The SM8850P Profiler Wrapper API

A dispatch, counter and measurement interface for the four compute engines of the Qualcomm Snapdragon 8 Elite Gen 5 (SM8850P)

Snapdragon Profiler project — `profiler.py`

Document revision 2.0

Abstract

`profiler.py` is a single-file, standard-library-only Python API that dispatches benchmark kernels to a Qualcomm HDK8850 board over `adb` and returns parsed, comparable measurements. It exposes twelve workloads across four compute engines (CPU, GPU, DSP, NPU) behind one uniform call; collects ARMv9 hardware performance counters alongside any CPU dispatch; runs converted neural networks through QNN; estimates total-board power from a calibrated model; and supports serial execution, genuine hardware-parallel execution, and dependency-ordered DAG execution.

This document is the complete reference: every class, every argument, every field, and the measurement caveats that decide whether a number you get back is trustworthy. It is written to be read in order once and then used as a lookup.

One standing warning about catalogues. The kernel and parameter tables reproduced here are hand-maintained and can lag the code. `python profiler.py --catalogue` and `python profiler.py --list-counters` are *generated from the registry at runtime* and are therefore always current. Where this document and those commands disagree, the commands are right.

Contents

1 Overview **4**
- 1.1 What this is . . . 4
- 1.2 The hardware it targets . . . 4
- 1.3 Why it is called an API . . . 4
- 1.4 Design commitments . . . 5
- 1.5 What it deliberately does not do . . . 5

2 Installation and prerequisites **5**
- 2.1 Requirements . . . 5
- 2.2 Provisioning the device . . . 6
- 2.3 Smoke test . . . 6

3 Quick start **7**

4 Execution model **7**
- 4.1 The path of one dispatch . . . 7
- 4.2 Backend command construction . . . 8
- 4.3 Privilege: why counters run under `su 0` . . . 8
- 4.4 Serial, parallel, and DAG semantics . . . 9

5 API reference **9**
- 5.1 `Backend` . . . 9
- 5.2 `Task` . . . 10
- 5.3 `CPU affinity: the core argument` . . . 12

5.4 `Threading: the threads argument` . . . 13
5.5 `Hardware counters: counters, per_core, counter_cpus` . . . 13
5.6 `Effective clock and throttling detection` . . . 16
5.7 `Profiler` . . . 16
5.8 `Thermal reporting` . . . 17
5.9 `Result` . . . 19
5.10 `Device` . . . 20
5.11 `build_command(task, base) -> str` . . . 21
5.12 `The registry: WORKLOADS, Workload, KernelEntry` . . . 21

6 Kernel and variant catalogue **22**

7 Parameter reference **23**
7.1 How keywords become flags . . . 23
7.2 Parameter renaming: `param_map` . . . 24
7.3 Parameters accepted by each kernel . . . 24
7.4 Derived FLOP and byte counts follow *your* parameters . . . 25
7.5 Iteration accounting, and why it is not obvious . . . 26

8 What the fields mean, per backend **27**
8.1 Selection of `compute_us` . . . 27
8.2 Transport and overhead fields . . . 27
8.3 What counters mean on a non-CPU backend . . . 28
8.4 Reading a result correctly . . . 28

9 Your own kernels: `custom` **28**

10 Converted networks: `model` **29**
10.1 The model registry . . . 29
10.2 `model(backend, name, **params)` . . . 30
10.3 `time_model(backend, name, reps=100)` . . . 30

11 Power estimation: `power()` **30**
11.1 Three things this is not . . . 31
11.2 Arguments . . . 31
11.3 What each engine returns . . . 31
11.4 Board transfer . . . 31
11.5 Why there is no multi-engine power call . . . 32
11.6 Operational notes . . . 32

12 Command-line interface **32**
12.1 `--repeat`: process repeats, not kernel iterations . . . 33

13 Recipes **33**
13.1 One roofline point on every engine . . . 33
13.2 A parameter sweep written to CSV . . . 33
13.3 Contention: the same batch, both ways . . . 34
13.4 CPU cluster and core study, with counters . . . 34
13.5 Thread scaling, done so the answer means something . . . 34
13.6 Scalar versus vectorised, on one engine . . . 34
13.7 A pipeline expressed as a DAG . . . 34
13.8 Comparing engines on one network . . . 35
13.9 Checking what will run, without a board . . . 35

13.10A guarded sweep that reports its own failures 35

14 Extending the wrapper **35**
14.1 Running a kernel without touching this file 35
14.2 Adding a variant to an existing kernel 35
14.3 Adding a backend to an existing workload 36
14.4 Adding a whole workload 36
14.5 Adding a new timing field 36
14.6 Adding a derived counter metric 36

15 Semantics, caveats, and measurement validity **36**
15.1 The caveats that change conclusions 36
15.2 Checking a number before you quote it 37

16 Troubleshooting **37**

A Counter catalogue **38**
A.1 Derived metrics 39

B Glossary of the numbers this document quotes **39**

1 Overview

1.1 What this is

The Snapdragon Profiler project builds and characterises benchmark kernels for the SM8850P SoC. Underneath, every measurement is an on-device binary invoked over `adb` with a backend-specific working directory, library path, affinity mask, privilege wrapper and flag set, whose stdout is then scraped for timing fields that differ per backend.

`profiler.py` is the API over that machinery. It turns

```
adb shell "cd /data/local/tmp/profiler-kernels/npu && LD_LIBRARY_PATH=... \
  ADSP_LIBRARY_PATH=... ./conv_qnn_fp16 --perf-mode sustained --reps 5 --cin 8 --cout 8"
```

into

```
p.run(convolution(Backend.NPU, cin=8, cout=8))
```

and returns a `Result` whose `compute_us`, `gflops` and `gbps` fields mean the same thing regardless of which engine produced them.

1.2 The hardware it targets

The course hardware is the **HDK8850**, and several numbers in the API are properties of that part rather than of the software.

Unit	What the API assumes about it
SoC	SM8850P, "Snapdragon 8 Elite Gen 5", codename `canoe`, SoC ID 661. Android 16, userdebug, `su 0` available.
CPU	Oryon-2, 6 Gold + 2 Prime. **Gold is cores 0–5** (28-point OPP ladder topping out at 3.6288 GHz); **Prime is cores 6–7** (27 points to 4.3968 GHz). The two ladders share only four setpoints. These ceilings are hard-coded as `CLUSTER_MAX_GHZ` and used to flag under-clocked runs (§5.6).
GPU	Adreno 840, OpenCL, 18 power levels from 160 MHz to 1.2 GHz. GPU hardware perfcounters are *not* readable on this build; the GPU reports `CL_PROFILING` durations only.
DSP	Hexagon cDSP v81, 8 HVX contexts, 8 MB VTCM, reached over FastRPC.
NPU	Hexagon HTP, driven through QAIRT/QNN 2.40. Shares the Hexagon subsystem with the DSP, which has consequences for parallelism (§4.4).

The Prime ceiling is 4.3968 GHz, not 4.608. The sysfs node `cpuinfo_max_freq` reports 4608000 and the governor will accept it, but that setpoint is absent from `scaling_available_frequencies`, the effective clock only reaches about 3.8–3.9 GHz there, and under sustained load the governor settles to 4.34. 4.3968 is the fastest setpoint that can actually be selected and held, so it is the one the API compares against.

1.3 Why it is called an API

The name is accurate in the ordinary sense: it is a programmatic interface with a small, stable vocabulary of objects (`Task`, `Result`, `Profiler`, `Device`, `Backend`) that hides a transport and a parsing layer behind method calls. Callers write Python against those objects; they never construct shell strings, never touch `adb`, and never write a per-backend output parser. Swapping the engine a workload runs on is a one-token change, which is precisely the property an interface is supposed to give you.

It is a *host-side* API: it is imported into a Python process on your workstation, and the boundary it wraps is the USB link to the board, not a network service. There is no server and no HTTP.

1.4 Design commitments

Uniform, not lossy.
Every backend reports through the same `Result` shape, but nothing is averaged away or hidden. Every `key=value` token the kernel printed survives in `Result.fields` and the untouched stdout survives in `Result.raw`.

Tool, not agent.
The API does exactly what it is told. It never picks a variant for you, never retries, never warms up behind your back, never tunes, and never flags an anomaly. Defaults exist so a call is short; they are documented here, printed by `--catalogue`, and visible in the command it builds (`verbose=True`).

Costs stay visible.
Backend-native compute time and host-side transport are reported as separate fields. The NPU's graph-compile time, the GPU's host-to-device copy and the DSP's FastRPC marshalling are not folded into the headline number.

Real concurrency.
Parallel execution is genuine simultaneous hardware use, not interleaving. See §4.4.

Fail early, on the host.
A mistyped variant, an impossible size, or `threads=` on a kernel that cannot take it raises `ValueError` at `Task` construction — not on the device three dispatches into a sweep. The messages name the valid alternatives.

Refuse rather than guess.
Where a number cannot be produced honestly the API says so. `power()` returns `available: False` with a reason for the DSP rather than inventing watts; a counter this silicon does not implement raises instead of returning the silent zero the hardware would give you.

Standalone.
The module depends only on the Python standard library and an `adb` on `PATH`. It does not require the control container, the storage stack, or the orchestrator.

1.5 What it deliberately does not do

It does not build or push kernels (see §2), does not persist results to the project's Parquet/SQLite store, does not capture Perfetto traces, does not read GPU, DSP or NPU hardware counters (only CPU PMU counters, §5.5), and does not measure energy directly — there is no readable power rail on this board, so `power()` is a fitted model and says so (§11). It is the dispatch and measurement front door; the heavier subsystems live elsewhere in the repository.

2 Installation and prerequisites

2.1 Requirements

Requirement	Notes
Python 3.10 or newer	The module uses PEP 604 unions (`float | None`) and builtin generics. 3.9 will fail to import. No `pip install` is needed: the module imports only from the standard library.
`adb` on `PATH`	Any recent platform-tools. Verify with `adb devices`; exactly one device should be listed, or pass a serial explicitly. Version 30 or newer is recommended — an `adb` older than the device's `adbd` negotiates badly.
An HDK8850 (SM8850P) board	Connected over USB-C, in `device` state, with USB debugging authorised. TCP transport works too if `adb connect` has already been done.
Kernels present on device	The prebuilt binaries must exist under `/data/local/tmp/profiler-kernels`.
`su 0` on the device	Required *only* for hardware counters (§5.5). Everything else runs at shell uid.

2.2 Provisioning the device

The wrapper assumes the kernel tree already sits on the board. Push it with the project's provisioner, which also verifies five functional gates before it reports success:

```
python scripts/provision_device.py \
    --qairt-root "<path>/qairt/2.40.0.251030" \
    --dsp-skel   kernels/matmul/dsp/prebuilt/libdsp_matmul_skel.so
python scripts/verify_board.py        # expect: 0 fail -> BOARD OK.
```

To go the other way — put the board back to unprovisioned, so the setup path can be tested from scratch — use `scripts/reset_device.py`. It reports what it would delete and requires `--yes` before removing anything. **Rebooting does not do this**: `/data` is f2fs on a real block device, so `/data/local/tmp` survives a reboot and the kernels are still there afterwards. Provisioning is idempotent: re-running after a failure is the normal remedy.
The expected on-device layout is:

```
/data/local/tmp/profiler-kernels/
    matmul_neon_handtuned, matmul_eigen_baseline   <- CPU matmul at the base
    gpu/         matmul_opencl_*                   + libs (LD_LIBRARY_PATH=".")
    dsp/         *_dsp*, stubs and skels           (FastRPC)
    npu/         *_qnn_*                           + QNN backend libraries
    saxpy/       saxpy_scalar, saxpy_neon, saxpy_opencl_fp32
    reduction/   reduction_scalar, reduction_neon, reduction_opencl_fp32
    convolution/ conv_scalar, conv_neon, conv_int8_neon, conv_opencl_fp32
    nbody/       nbody_neon, nbody_opencl_fp32
    decode/      decode_neon, decode_opencl_fp32, decode_opencl_gemv
    cachelat/    cache_latency
    memlab/      gpu_memlab
    custom/      whatever you build yourself
/data/local/tmp/
    mnv2/  nlp/  qwen/  llama/  sd15/              <- converted networks (DLCs)
```

If a binary is missing, the corresponding call fails with a shell error in `Result.error` rather than raising. Networks are staged separately and are not part of provisioning; see §10.

2.3 Smoke test

```
python profiler.py --demo
```

This runs a short live tour: a connectivity check, one convolution on all four backends in parallel, an NPU roofline inversion pair, a CPU affinity comparison, a hardware-counter comparison at one and six threads, and a small dependency DAG. If the demo prints results, the API is correctly wired to your board.

3 Quick start

```
from profiler import Profiler, Backend, matmul, saxpy, reduction, convolution

p = Profiler()                                  # uses the only attached device

# 1. One dispatch.
r = p.run(convolution(Backend.NPU, cin=8, cout=8))
print(r.summary())
print(r.compute_us, r.gflops, r.fields["finalize_us"])

# 2. The same workload on every engine, at the same time.
for r in p.run_parallel([convolution(b, cin=8, cout=8) for b in Backend]):
    print(r.summary())

# 3. The same batch, strictly one at a time (clean, uncontended numbers).
for r in p.run_serial([convolution(b, cin=8, cout=8) for b in Backend]):
    print(r.summary())

# 4. Hardware counters riding along with a CPU dispatch.
r = p.run(convolution(Backend.CPU, cin=32, cout=32,
                      threads=6, core="gold",          # where, and how many
                      counters="cache", per_core=True))
print(r.derived["ipc"], r.derived["l1d_miss_pct"])
print(r.counter_table())

# 5. Dependency order: warm up on CPU, then fan out to GPU and NPU.
results = p.run_dag([
    reduction(Backend.CPU, size=1 << 20, name="warmup"),
    convolution(Backend.GPU, cin=16, cout=16, name="gpu", depends_on=["warmup"]),
    convolution(Backend.NPU, cin=16, cout=16, name="npu", depends_on=["warmup"]),
])
print(results["npu"].summary())
```

Everything else in this document is detail on those five calls, plus two subsystems that stand slightly apart: converted networks (§10) and power estimation (§11).

4 Execution model

4.1 The path of one dispatch

```
Task(workload, backend, params, variant, core, threads, counters, ...)
      |
      |  1. registry lookup: WORKLOADS[workload].kernels[backend] -> KernelEntry
      |  2. variant -> binary name (or a --variant flag, DSP and nbody-GPU)
      |  3. entry.defaults overlaid with task.params       (task wins)
      |  4. entry.param_map renames keys the kernel spells differently
      |  5. params -> CLI flags   (perf_mode="sustained" -> --perf-mode sustained)
      |  6. backend prologue: cd / LD_LIBRARY_PATH / ADSP_LIBRARY_PATH / taskset
      |  7. if counters: prepend `simpleperf stat --csv -e ...`
      |                  and wrap the WHOLE command in `su 0 sh -c '...'`
      v
build_command(task, base)  ->  one shell string
      |
      v
```

```
Device.shell(cmd, timeout)  ->  adb [-s SERIAL] shell "<cmd>"   (blocking)
      |
      v
stdout scraped twice:  key=value / "key: value"  -> Result.fields
                       simpleperf --csv rows      -> Result.counters/.per_core
      |
      v
Result(ok, compute_us, wall_us, gflops, gbps, fingerprint, fields,
       counters, derived, per_core, raw)
```

One `Task` is exactly one process on the device. Nothing is batched, cached or reordered. The two parsers read the same stream without colliding, because simpleperf's CSV rows are comma-separated and the kernels' output is not.

4.2 Backend command construction

Each engine needs a different environment. The API applies it for you:

Backend	Command shape
CPU	`taskset <mask> <base>/<subdir>/<binary> <flags>` The affinity mask comes from `Task.core` (§5.3).
GPU	`cd <dir> && LD_LIBRARY_PATH=<ldpath> ./<binary> <flags>` Adreno matmul and decode ship their libraries beside the binary (`ldpath="."`); the other GPU kernels use `/vendor/lib64`.
DSP	`cd <dir> && LD_LIBRARY_PATH=. ADSP_LIBRARY_PATH='.;<dir>' ./<binary> <flags>` FastRPC needs the stub next to the working directory and the skel on the DSP's library path.
NPU	`cd <dir> && LD_LIBRARY_PATH=<dir> ADSP_LIBRARY_PATH=<dir> ./<binary> <flags>` QNN resolves its HTP backend libraries from those paths.
model	`cd <modeldir> && rm -rf <out> && LD_LIBRARY_PATH=<modeldir> ... ./qnn-net-run --dlc_path ... --backend libQnn{Cpu,Gpu,Htp}.so ...` Networks bypass the kernel tree entirely (§10).

Pass `verbose=True` to the `Profiler` to print each command before it runs — the fastest way to confirm what the API actually did — or call `build_command` to get the string without a board attached.

4.3 Privilege: why counters run under `su 0`

When, and only when, counters are requested, the entire command is re-wrapped:

```
su 0 sh -c '<the command that would otherwise have run>'
```

On this board the shell domain cannot open the PMU. `simpleperf` answers *"Event type 'cpu-cycles' is not supported on the device"* for every event, including ones `simpleperf list` advertises, while the identical command under `su 0` counts normally. `perf_event_paranoid` is already `-1`, so this is an SELinux domain restriction, not a paranoia level.

This is not `adb root`, and the difference matters. `su 0` elevates one command from an unrooted `adbd`. `adb root` restarts `adbd` itself in a domain that cannot reach the OpenCL driver, which breaks every GPU dispatch — `clGetPlatformIDs` returns `-1001`, no platform. Never run `adb root` on this board while profiling; the API is careful not to need it.

4.4 Serial, parallel, and DAG semantics

Serial Tasks run strictly one after another. Nothing else the API controls is on the machine while a measurement is taken. Use this for any number you intend to quote.

Parallel Every task is launched on its own host thread, so N blocking `adb shell` invocations are in flight at once. The CPU cluster, the Adreno GPU and the Hexagon subsystem are independent hardware, so tasks on those genuinely execute simultaneously. Tasks on the *same* backend contend — that contention is real and measurable, and is often the point, but it is not a clean single-engine number.

One exception, and it is not visible from the host: NPU and DSP do not run in parallel. They are two software paths — QNN HTP and FastRPC — onto the *same* Hexagon cDSP, so they serialise in hardware. Submitting both concurrently looks correct from here: the dispatches go out together and their host-side windows overlap. The cDSP runs them one after the other anyway. Measured in Discovery 50, which found NPU+DSP to be the sole HW-SERIALIZE pair in the nine-pair matrix; first caught when a `wait_any([dsp, npu])` returned the DSP result while the NPU sat blocked behind cDSP warm-up. Effective width is four independent paths, or three whenever NPU and DSP are both in play. Do not read an NPU+DSP parallel batch as a speed-up measurement.

The same sharing cuts the other way. A busy NPU raises the Hexagon DVFS operating point, and the DSP — which cannot vote for its own clock on this board (§15.1) — is carried along with it. A DSP kernel measured next to an NPU peer can be *faster* than the same kernel measured alone (Discovery 75: 1.59×). Every DSP figure in this project is therefore the idle-Hexagon case, and "isolation" is not the cleanest measurement on this engine.

DAG Tasks are grouped into levels: every task whose dependencies have completed runs in the same parallel batch, then the next level starts. A task never starts before its dependencies finish. Cycles and unknown dependency names raise `ValueError` before anything is dispatched.

Wall-clock overlap in parallel mode is host-scheduled, so start times are not synchronised to the microsecond. For tightly-coupled interference studies, use longer per-kernel iteration counts so the overlap window dominates the launch skew.

5 API reference

5.1 `Backend`

An `Enum` of the four compute engines. `str(Backend.CPU) == "cpu"`, and the enum is iterable, so `for b in Backend` sweeps all four.

Member	Value	Hardware and native timing source
`Backend.CPU`	`"cpu"`	Oryon-2 cores, affinity via `taskset`; self-timed `wall_us` or `mean_us`.
`Backend.GPU`	`"gpu"`	Adreno 840, OpenCL; `CL_PROFILING` `duration_ns`.
`Backend.DSP`	`"dsp"`	Hexagon cDSP v81, FastRPC + HVX; on-DSP `dsp_us`.
`Backend.NPU`	`"npu"`	Hexagon HTP via QNN; `execute_us_mean` plus a separate `finalize_us`.

Not every workload exists on every backend. `nbody` is CPU and GPU only; `cachelat` is CPU only; `gpu_memlab` is GPU only; `decode` and `model` skip the DSP. Asking for a pair that does not exist raises `ValueError` naming the backends that do.

5.2 `Task`

A description of one dispatch. Constructing a `Task` performs no I/O; it validates the workload-/backend pair, the variant, the size, the thread request and the counter selection, then assigns a name.

```
Task(workload: str,
     backend: Backend,
     params: dict = {},
     variant: str | None = None,
     name: str | None = None,
     timeout: float = 180.0,
     depends_on: list[str] = [],
     core: object = None,
     threads: int | None = None,
     counters: object = None,
     per_core: bool = False,
     counter_cpus: object = None,
     binary: str | None = None)
```

Field	Type	Meaning
`workload`	`str`	A key of `WORKLOADS`: `"matmul"`, `"saxpy"`, `"reduction"`, `"convolution"`, `"nbody"`, `"decode"`, `"cachelat"`, `"morph"`, `"gpu_memlab"`, `"custom"`, `"model"`. Unknown values raise `ValueError` listing the known ones.
`backend`	`Backend`	Which engine. A pair with no kernel raises `ValueError` naming the backends that workload does have.
`params`	`dict`	Kernel parameters, passed through as CLI flags (§7).
`variant`	`str | None`	Which kernel implementation (§6). `None` selects the entry's default. A name that does not exist raises `ValueError` listing every valid variant, the default, and — when the typo is close to a real name — a "did you mean" line.
`name`	`str | None`	Identifier used by `run_dag` and printed by `Result.summary()`. Defaults to `"<workload>.<backend>[.<variant>]"`. Must be unique within a DAG.
`timeout`	`float`	Seconds before the `adb` call is abandoned and the task fails with exit code 124. Raise it for NPU work (§15).
`depends_on`	`list[str]`	Names of tasks that must complete first. Only consulted by `run_dag`.
`core`	see §5.3	CPU affinity: *where* the work is allowed to run. Ignored on non-CPU backends.
`threads`	`int | None`	*How many* worker threads the kernel spawns, passed as `--threads N`. Only accepted by threaded variants (§5.4).
`counters`	see §5.5	CPU hardware counters to collect alongside the dispatch. `None` means none.
`per_core`	`bool`	Break the counter totals down per CPU (`simpleperf --per-core`). Only meaningful with `counters`.

Field	Type	Meaning
`counter_cpus`	`int | list | str | None`	Count *per-CPU and system-wide* on these CPUs instead of per-process. See §5.5.
`binary`	`str | None`	Dispatch this device binary instead of looking one up in the registry. Used by `custom()` (§9).

Read-only properties

Property	Value
`task.entry`	The `KernelEntry` this task resolves to: device subdirectory, variant table, always-on flags.
`task.resolved_variant`	The variant that will actually run — `variant` if given, else the registry default. Worth checking: defaults are deliberately *not* the fastest variant, so that the naive/optimised gap stays visible.
`task.counter_events`	The flat, deduplicated event list that `counters` resolves to. Recomputed on every access, so assigning `task.counters = "stalls"` after construction takes effect.
`task.cores`	The list of CPU ids the affinity mask actually permits, e.g. `[6, 7]` for `core="prime"`.
`task.oversubscribed`	`True` when `threads` exceeds `len(task.cores)` — more workers than cores they are allowed on. This is legal and measurable, not an error.

What is validated at construction

All five checks run before any I/O, so a sweep fails on line one rather than twenty minutes in:

1. the workload exists;
2. the workload has a kernel on that backend;
3. the variant exists on that entry;
4. `size` matches `fixed_size` when the chosen variant is compiled for exactly one shape (§6);
5. `threads` is ≥ 1 *and* the resolved variant is in `threaded_variants`;
6. every requested counter is implemented on this silicon.

Convenience constructors

Eight module-level functions build a `Task` without naming the workload string. Task-level keywords (`name`, `variant`, `depends_on`, `timeout`, `core`, `threads`, `counters`, `per_core`, `counter_cpus`) are recognised and peeled off; *every other keyword becomes a kernel parameter.*

```
matmul(backend, **params)          -> Task   # dense square GEMM
saxpy(backend, **params)           -> Task   # y = a*x + y
reduction(backend, **params)       -> Task   # sum of floats
convolution(backend, **params)     -> Task   # 2D conv
nbody(backend, **params)           -> Task   # direct-summation N-body (CPU/GPU)
decode(backend, **params)          -> Task   # weight-stationary GEMM
custom(backend, binary, **params)  -> Task   # a kernel you wrote
model(backend, name, **params)     -> Task   # a converted network
```

```
# Equivalent:
saxpy(Backend.CPU, size=1 << 20, variant="scalar", core="prime", name="s1")
Task("saxpy", Backend.CPU, {"size": 1 << 20}, "scalar", "s1", core="prime")
```

Constructor	What the workload is for
`matmul`	Dense square matmul, `size=N` (default 512). Arithmetic intensity $\approx N/6$: the compute-bound end of the set. Variants differ enormously — GPU `best` is about 16× `naive`, CPU `eigen` about 3× `neon`.
`saxpy`	$y = ax + y$ over `size` floats (default 1M). AI 0.17 — the memory-bound end, and the workload where one CPU cluster beats the NPU by about 50×.
`reduction`	Sum of `size` floats (default 4M). Memory-bound like saxpy, *but* the NPU has a native `ReduceSum` so it compiles in milliseconds rather than seconds. The saxpy/reduction pair is a controlled experiment in operator support, holding arithmetic intensity roughly fixed.
`convolution`	2D convolution over `cin`/`cout`/`H`/`W`/`k`. AI ≈ 18 at $C = 8$, enough to feed the NPU's tensor cores: it wins by about 68× over one CPU cluster.
`nbody`	Direct-summation N-body, $O(N^2)$ arithmetic on $O(N)$ data, so AI $= 1.25N$ and *grows with the problem*. QNN cannot express it, so this is the one workload where the GPU competes on equal terms. CPU and GPU only, by design rather than omission.
`decode`	Weight-stationary GEMM $C[S \times n] = A[S \times k] \cdot W[k \times n]$ with the token count `tokens` exposed. That one knob walks the workload across the roofline ridge: $S = 1$ is LLM decode (every weight read once, DRAM-bound), large S is prefill (weights reused, compute-bound). The ranking inverts across it — at $S = 1$ one CPU core beats the NPU; at $S = 256$ the NPU wins 29× (Discovery 66).
`custom`	Your own kernel, by device filename. §9.
`model`	A converted network through `qnn-net-run`. §10.

Unrecognised keywords silently become flags. A typo such as `sizee=1024` produces `--sizee 1024` on the device rather than a Python error. Most kernels reject unknown flags and the task then fails, so check `Result.error` first when a call fails unexpectedly — but some kernels ignore them, in which case you have quietly measured the kernel's *default* instead of what you asked for. There is no way for the host to tell those two apart, which is why `verbose=True` is worth having on while you develop a sweep.

5.3 `CPU affinity: the core argument`

`core` decides *where* work may run. It accepts four forms and is resolved to a `taskset` hexadecimal mask.

Form	Mask	Meaning
`None` (default)	`f0`	Cores 4–7. This is what the project's sweep scripts use, so defaults reproduce the published roofline numbers. **Read the note below before relying on it.**
`core=7`	`80`	That one core.
`core=[6, 7]`	`c0`	Those cores, OR-ed together.
`core="prime"`	`c0`	Prime cluster, cores 6–7, up to 4.3968 GHz.
`core="gold"`	`3f`	Gold cluster, cores 0–5, up to 3.6288 GHz.

Form	Mask	Meaning
`core="all"`	`ff`	All eight cores.
`core="0xc0"` / `"c0"`	as given	Raw mask, passed through unchanged and not validated.

The default mask straddles the cluster boundary. On this SoC Gold is cores 0–5 and Prime is 6–7, so `f0` (cores 4, 5, 6, 7) permits *two Gold cores and two Prime cores*. The scheduler may place a single-threaded kernel on either, and two runs with identical arguments can land on different clusters with a 1.13× clock difference between them. Two consequences:

- For a number you intend to quote, name the cluster explicitly: `core="prime"` or `core="gold"`.
- `clock_of_max_pct` and `below_max_clock` (§5.6) are computed *only* when the mask lies entirely inside one cluster, because only then is there an unambiguous ceiling to compare against. Under the default mask they are silently absent.

Cluster membership is fixed for the SM8850P; masks are not re-derived at runtime.

5.4 `Threading: the threads argument`

`threads=N` passes `--threads N` to the kernel, asking it to spawn N workers. It is independent of `core`, which only decides which CPUs those workers are *allowed* on. Both are recorded, and the combination is free: six threads on a one-core mask is a legal, measurable experiment — `task.oversubscribed` reports `True` and the kernel time-slices.

Threading is per-variant, not per-workload. The binary either accepts the flag or hard-errors on it, and variants inside one registry entry differ: `conv_neon` is threaded, `conv_scalar` sits in the same entry and rejects `--threads`. Asking for threads on a variant that cannot take them raises at construction and names the ones that can.

Workload	Threaded variants (CPU)
`matmul`	`neon`. *Not* `eigen`.
`saxpy`	`scalar`, `neon`
`reduction`	`scalar`, `neon`
`convolution`	`neon`, `int8`. *Not* `scalar`.
`nbody`	`neon`

A trap worth knowing about matmul. The CPU matmul spawns and joins its threads *inside* the timed region, so a single-iteration run charges the whole thread-spawn cost to a short measurement — and the shorter the region, the worse parallelism looks. Measured on Gold, matmul $N = 512$ scales 2.55× at one iteration and 4.11× at thirty. The registry therefore sends `--iterations 10` by default for CPU matmul. If you are studying parallel efficiency, raise it further; if you lower it to 1 you will measure thread creation and call it poor scaling.

5.5 `Hardware counters: counters, per_core, counter_cpus`

Any CPU dispatch can carry ARMv9 PMU counters, collected by `simpleperf` wrapped around the same command.

The prefix is applied on *every* backend, not only the CPU, and a GPU, DSP or NPU dispatch with counters succeeds. What comes back is the counters of the extbfARM process driving the accelerator, not of the accelerator — which is a useful measurement, and not the one people assume. See §efsec:accelcounters. There is no way to read Adreno or Hexagon hardware counters from this board at all.

```
r = p.run(convolution(Backend.CPU, cin=32, cout=32, core="gold", threads=6,
                      counters="cache", per_core=True))

r.counters    # {'cpu-cycles': 1.9e10, 'raw-l1d-cache': 1.2e8, ...}  raw counts
r.derived     # {'ipc': 2.91, 'l1d_miss_pct': 0.28, 'effective_ghz': 3.41, ...}
r.per_core    # {0: {...}, 1: {...}, ...}  one dict per CPU
print(r.counter_table())
```

`counters` — what to collect

Accepts `None`, a preset name, a single event name, or a list mixing the two. Presets expand; duplicates collapse; order is preserved.

Preset	Events	What it answers
`basic`	2	Cycles and instructions — IPC, and nothing else. The cheapest way to get `effective_ghz`.
`branch`	7	Prediction accuracy, split by branch kind: direct, indirect (the expensive kind), and returns.
`cache`	10	L1D, L1I and L2 accesses, refills and writebacks. The default choice for a memory-hierarchy question.
`tlb`	10	L1/L2 TLB accesses and refills, plus page-table walks.
`stalls`	8	Frontend vs backend stalls, in both cycles and issue slots, plus memory-attributed backend stalls.
`mix`	9	Instruction mix: retired vs speculative, loads, stores, integer, and FP/SIMD.
`memory`	9	Memory and bus accesses with the refills and memory stalls that explain them.
`prefetch`	9	Cache traffic split by who generated it — the program or the hardware prefetcher.

`python profiler.py --list-counters` prints all eight presets, all 92 verified events grouped by subsystem, and all 43 blocked ones with the reason for each.

The 12-counter limit

The Oryon-2 PMU has **12 hardware counters per core** (`MAX_HW_COUNTERS`). Up to twelve events are collected simultaneously and exactly. Ask for more and `simpleperf` time-multiplexes them: each event is counted for part of the run and scaled up, so the numbers become estimates. The API does not stop you — multiplexing is sometimes the right trade — but it sets `Result.multiplexed = True` and `summary()` prints `[multiplexed]`. Every shipped preset is ≤ 12 events, so presets never multiplex.

Blocked counters, and why refusing beats returning zero

`simpleperf list raw` advertises 742 events on this part and *none of them error*. The ones the silicon does not implement return a silent, confident **zero**. A zero is indistinguishable from a real measurement of nothing, so it is the most dangerous possible failure mode: you can build an entire argument on a counter that was never counting.

The module therefore ships a whitelist. 92 events were each run against a memory-heavy workload on both clusters and returned non-zero; 43 more returned exactly zero and are blocked. Requesting a blocked event raises `ValueError` naming the reason:

```
ValueError: counter 'raw-l3d-cache' reads a constant 0 on this board -- no
L3/system-cache PMU: the SLC is invisible to the core PMU on this SoC (sysfs
exposes only L1D/L1I/L2). It would not error on device, it would silently
return 0.
```

The four blocked families, and what to use instead:

Blocked family	Reason, and the substitute
`raw-l3d-*`, `raw-ll-*`	There is no L3/system-cache PMU: the SLC is invisible to the core PMU. L2 refills (`raw-l2d-cache-refill`) are the last visible level.
`raw-stall-backend-l1d`, `-l2d`, `-tlb`, and the frontend equivalents	Per-structure stall attribution is not implemented. Use `raw-stall-backend-mem` for memory-attributed stalls.
`raw-l1d-cache-hit`, `raw-l2d-cache-hit`, `raw-l1d-cache-allocate`	Cache *hit* counters are not implemented. Derive hits as (accesses − refills).
Prefetch refill/hit attribution (`*-refill-hwprf`, `*-hit-rd-f*`)	Only the prefetcher's *access* counters work (`raw-l1d-cache-hwprf`, `raw-l2d-cache-hwprf`). You can count prefetcher traffic; you cannot measure its hit rate.
`raw-ase-inst-spec`	Reads 0 even on NEON code. SIMD counts under `raw-vfp-spec`.

`per_core` — the per-CPU breakdown

`per_core=True` adds `--per-core` to `simpleperf`, which reports each CPU separately instead of one aggregate. `Result.per_core` becomes `{cpu_id: {event: count}}`, and because `simpleperf` prints no aggregate row in this mode, `Result.counters` is reconstructed by summing the cores.

This is how you see load imbalance. Six threads with one straggler look identical in the totals and obvious per core.

`counter_cpus` — process-scoped vs system-wide

By default counting is **process-scoped**: only your workload is counted, wherever the scheduler puts it. That is what you want almost always.

`counter_cpus` switches to `simpleperf --cpu`, which counts **per-CPU and system-wide** on the named CPUs — so it also catches whatever else Android decides to run there. It accepts an int, a list of ints, or a raw string such as `"0-5"`. Use it when the question is "what is happening on these cores" rather than "what is my process doing"; interpret the result knowing that the answer includes other people's work.

What counters cost

`simpleperf` itself costs nothing measurable. The governor sampler that rides along with it (§5.6) is not free: it forks `cat` and `sleep` every 2 ms, which is cheap when cores are idle and expensive when they are not — measured at +1.5% on one Prime core with the machine otherwise idle, and +31% on a six-thread convolution occupying the whole Gold cluster. That perturbation would land squarely on the multi-core scaling experiments and read as poor parallel efficiency, so **the sampler only runs for single-threaded dispatches**. With `threads > 1` you get counters without `scaling_cur_freq_ghz`.

5.6 `Effective clock and throttling detection`

Two different clock numbers can appear in `Result.derived`, and they routinely disagree. Both are present only when counters were requested.

Field	What it is
`effective_ghz`	**Measured.** Cycles actually retired, divided by `threads`, divided by `simpleperf`'s own *Total test time*. This is the clock the silicon really ran at.
`scaling_cur_freq_ghz`	**Requested.** The maximum value the governor asked for, sampled from sysfs every 2 ms *during* the run.
`clock_of_max_pct`	`effective_ghz` as a percentage of the cluster ceiling.
`scaling_cur_freq_samples`	How many governor samples were taken; reported so you can judge the one above.

`Result.below_max_clock` is `True` when `effective_ghz` falls below `CLOCK_FLAG_RATIO` (0.80) of the cluster ceiling, and `summary()` then appends `[clock NN% of max]`.

This flag does not mean thermal throttling. Under the shipping `walt` governor a low clock is ordinary DVFS. It means "this did not run at full speed"; whether that was intended is yours to judge. What it reliably catches is a sweep that silently drops from 4.40 to 1.37 GHz half way through, which otherwise reads as a workload effect.

Three constraints on these fields, each the result of getting it wrong first:

- **The denominator must be `simpleperf`'s window, not the kernel's `wall_us`.** The counts cover the whole process — startup, input generation, teardown — while `wall_us` covers only the timed region. Dividing whole-process cycles by region-only time reported 17 GHz on a 4.4 GHz part.
- **`scaling_cur_freq_ghz` needs a long enough run.** It is reported only when the workload ran at least 15 ms and produced at least five samples. A 7 ms workload yields one or two in-run samples out of thirty — the rest are idle before and after — and the maximum then reads *lower* than the effective clock, teaching the exact opposite of the truth. Gating on sample count does not catch this; gating on runtime does.
- **`clock_of_max_pct` needs a single-cluster mask.** With the default `core=None` it is absent (§5.3).

5.7 `Profiler`

The dispatcher.

```
Profiler(device: Device | None = None, *, verbose: bool = False,
         thermal: bool = True, thermal_limit_c: float = 90.0)
```

`device` defaults to `Device()`, which targets the only attached `adb` device. `verbose=True` prints each generated shell command to stdout before it runs. `thermal` and `thermal_limit_c` control the temperature check described in §5.8.

`run(tasks, *, parallel=False)`

The general entry point.

Call	Returns
`run(task)`	a single `Result`
`run([t1, t2, ...])`	`list[Result]`, executed serially, input order preserved
`run([t1, t2, ...], parallel=True)`	`list[Result]`, executed concurrently, input order preserved

The `parallel` flag is the whole difference between an isolated measurement and a contention measurement, which makes A/B studies a one-token edit:

```
clean     = p.run(batch, parallel=False)
contended = p.run(batch, parallel=True)
```

`run_serial(tasks) -> list[Result]`

Explicit serial form. Equivalent to `run(tasks)` on a list.

`run_parallel(tasks) -> list[Result]`

Explicit parallel form. Launches one host thread per task (`ThreadPoolExecutor` sized to the batch), so all dispatches are in flight simultaneously. Returns when the last one completes; results stay in input order. An empty list returns an empty list.

`run_dag(tasks) -> dict[str, Result]`

Executes named tasks in dependency order, one parallel level at a time, and returns a mapping from `Task.name` to `Result`.

Raises `ValueError` if names are not unique, if a `depends_on` entry names a task that is not in the list, or if the dependency graph contains a cycle. All three checks happen before any dispatch.

A failed dependency does *not* cancel its dependents: a task whose dependency returned `ok=False` still runs. Check `ok` on the upstream result if that matters to your experiment.

`time_model(backend, name, reps=100) -> dict`

Per-inference time for a converted network, with graph preparation removed. See §10.

`power(task, *, seconds=35, artifact=None) -> dict`

Estimated total-board power and energy while the task runs in a loop. See §11.

5.8 Thermal reporting

This HDK runs hot, and has fallen over: a sustained NPU graph reached 98 °C with mitigation engaged, and the board dropped off USB and needed its power cut by hand (Discovery 73). After every dispatch the profiler reads the die sensors, records them on the `Result`, and prints one line to stderr if the hottest zone has reached the limit (once per hot spell, again only if it climbs another 5 °C).

```
TOO HOT: 92 C on cpu-0-5-1 (limit 90 C). Let the board cool for a minute
before the next run -- measurements taken now are throttled, and this board
has dropped off USB at 98 C.
```

`summary()` also appends `[HOT 92C]`, and a later dispatch that has come back down prints `cooled to 71 C.` once.

It reports; it does not act

The profiler will not sleep, throttle, retry, re-order or refuse. Whether a hot board invalidates *your* measurement depends on what you are measuring — a thermal study wants the hot board — so that judgement stays with you. This is the same commitment as everywhere else in the API: it tells you what happened and changes nothing on your behalf.

Where 90 °C comes from

Measured on board 5cabf4fd, 2026-09-22:

State	Hottest zone
Idle	≈30 °C
One heavy 6-thread CPU dispatch	62 °C
Four back to back (the lab plateau)	77–79 °C
All-core interference (Task 7), just after the run	88–92 °C, falls within seconds
Runaway NPU (Discovery 73)	98 °C, board lost

90 sits in the gap: clear of the 78 °C an ordinary CPU sweep reaches and of the brief peaks after an all-core run, so it does not cry wolf, and still well under the 98 where this board has actually been lost. Change it with `thermal_limit_c`.

Which sensors, and the 105 °C trap

`/sys/class/thermal` exposes 135 entries on this board and **not all of them are sensors**. `cpu-hw-trip-0` and `cpu-hw-trip-1` read a constant `105000`: they are trip-point *settings*. Taking the maximum across every zone therefore reports 105 °C forever, on an idle board — a false claim this project published once before anyone checked the node type.

The selector is therefore a **whitelist**, `_THERMAL_ZONE_RE`, covering 40 zones: 16 CPU core sensors, 4 cluster-cache, 11 `gpuss-*`, 4 `nsphvx-*` (Hexagon HVX, the DSP), 4 `nsphmx-*` (Hexagon HMX, the NPU) and `ddr`. Everything else — trip points, battery-current-limit levels, the `Debug_Board` battery dummy, camera, modem and PMIC package sensors — is ignored. `verify_board.py` uses the same whitelist.

What it costs, and when to switch it off

Scanning all 135 zones costs about a second per read — two `cat`s per zone over `adb`. Instead the zone indices are resolved **once** (about 560 ms, on the first dispatch) and subsequent reads are a single `cat` of the 40 whitelisted files.

Setting	Cost, measured
`thermal=True` (default)	about **16 ms** added per dispatch in steady state, plus roughly 560 ms once on the first.
`thermal=False`	nothing. `temp_c` is `None` and `temps` is empty.

Worth switching off for a long sweep of very short dispatches, where 16 ms can exceed the kernel time and dominates the run. Leave it on for anything that loads the board for more than a moment — which is when it matters.

Reading it yourself

```
r = p.run(convolution(Backend.CPU, cin=64, cout=64, threads=6, core="gold"))
r.temp_c                                    # 78.7
r.too_hot                                   # False at the default limit
sorted(r.temps.items(), key=lambda kv: -kv[1])[:3]
# [('cpu-0-5-1', 78.7), ('cpu-0-5-0', 75.7), ('cpu-0-1-1', 75.7)]
```

5.9 `Result`

The parsed outcome of one dispatch.

Field	Type	Meaning
`task`	`Task`	The task that produced this result.
`ok`	`bool`	`False` on non-zero exit, on a kernel-reported status that is not one of `OK`/`SUCCESS`/`PASS`/`PASSED`/`DONE`, on timeout, or when the kernel printed no timing field at all.
`compute_us`	`float | None`	Backend-native compute time in microseconds (§8). This is the number to compare across engines.
`compute_key`	`str | None`	*Which* kernel field `compute_us` came from. Worth checking when comparing engines: the semantics differ per field.
`wall_us`	`float | None`	The kernel's own wall-clock figure where it prints one. On the DSP and NPU this includes host-side transport, so `wall_us` ≫ `compute_us` is expected and meaningful.
`gflops`	`float | None`	Derived: FLOP count divided by `compute_us`, with the iteration corrections of §7.5 applied.
`gbps`	`float | None`	Derived: byte count divided by `compute_us`. Present when the kernel prints `bytes=`, when the workload has a `bytes_fn`, or when the kernel prints its own `useful_gbps`.
`fingerprint`	`float | None`	The kernel's numerical checksum, used to confirm the computation was correct before trusting its timing. For the QNN matmul clients, which print `fingerprint_sum` and `fingerprint_frob` but never a combined line, the two halves are added.
`fields`	`dict`	Every `key=value` token and `key: value` line parsed from stdout, numerics coerced to `int`/`float`, plus `host_wall_us` added by the profiler.
`raw`	`str`	Unmodified stdout.
`error`	`str | None`	Failure text (first 300 characters of stderr, or stdout, or the exit code) when `ok` is `False`.
`counters`	`dict`	Event → raw count. Empty when no counters were requested.
`per_core`	`dict`	`{cpu_id: {event: count}}`. Empty unless `per_core=True`.
`derived`	`dict`	Ratios the raw counts imply — IPC, miss rates, stall percentages, effective clock. Only the ones the present events support.

Field	Type	Meaning
`multiplexed`	`bool`	More than 12 events were requested, so the counts are scaled estimates rather than exact.
`below_max_clock`	`bool`	Ran under 80% of the cluster ceiling (§5.6).
`temp_c`	`float | None`	Hottest die sensor immediately *after* the dispatch, in °C. `None` when thermal reporting is off (§5.8).
`temps`	`dict`	Every zone read, `{zone: degrees}`. A 78 °C core beside a 30 °C GPU is a different situation from both at 78.
`too_hot`	`bool`	`temp_c` reached the limit. Reported, never acted on.

Properties

Property	Value
`result.iterations`	How many times the kernel repeated the work inside one dispatch — read from `iterations/reps/inner-reps` in the output, but only when `compute_key` is a *total* field (`wall_us`). Otherwise 1.
`result.compute_us_per_iter`	`compute_us / iterations`: the per-dispatch latency.

`summary() -> str`

A one-line, column-aligned digest: task name; then either `N us` or `N us/iter xM` when the kernel repeated the work M times; then GFLOP/s, GB/s, any of `ns_per_access` / `finalize_us` / `transfer_ns` / `readback_ns` that are present, the fingerprint, IPC when counters ran, and the `[multiplexed]` / `[clock NN% of max]` markers. On failure it returns `"[FAIL] <name>: <error>"`.

It is a display helper — read the typed fields, not this string, when you are computing on the results.

`counter_table() -> str`

A multi-line block: every raw count, then the per-core matrix if there is one, then every derived ratio. Returns `"(no counters collected)"` when there are none.

5.10 `Device`

The `adb` transport.

```
Device(serial: str | None = None, adb: str = "adb",
       base: str = "/data/local/tmp/profiler-kernels")

device.shell(command: str, timeout: float = 180.0) -> tuple[str, str, int]
device.is_connected() -> bool
```

- `serial` selects one board when several are attached (`adb devices`); `None` uses the only one.
- `adb` lets you point at a specific platform-tools binary.

- `base` relocates the on-device kernel tree. It does *not* relocate converted networks, which live under `/data/local/tmp` independently.
- `shell` returns `(stdout, stderr, returncode)` and never raises: a timeout is reported as return code 124 with an explanatory stderr.
- `is_connected` runs `echo ok` with a 10-second timeout.

```
p = Profiler(Device(serial="5cabf4fd"), verbose=True)
if not p.device.is_connected():
    raise SystemExit("board not reachable")
```

5.11 `build_command(task, base) -> str`

Returns the exact device shell command a task would run, without running it. Useful for logging a sweep's provenance, for pasting a single case into an `adb shell` session, and for unit-testing flag construction with no hardware attached.

```
>>> from profiler import build_command, matmul, Backend, DEVICE_BASE
>>> print(build_command(matmul(Backend.DSP, size=512), DEVICE_BASE))
cd /data/local/tmp/profiler-kernels/dsp && LD_LIBRARY_PATH=.
ADSP_LIBRARY_PATH='.;/data/local/tmp/profiler-kernels/dsp'
./matmul_dsp_hvx --size 512

>>> print(build_command(matmul(Backend.GPU, variant="best"), DEVICE_BASE))
cd /data/local/tmp/profiler-kernels/gpu && LD_LIBRARY_PATH=.
./matmul_opencl_fp32_3x16u4_sg_half
```

(Line breaks above are for the page; the real return value is one line.) With counters requested the returned string additionally carries the `simpleperf` prefix, the governor sampler, and the surrounding `su 0 sh -c '...'` wrapper.

5.12 The registry: `WORKLOADS`, `Workload`, `KernelEntry`

`WORKLOADS` is a module-level dictionary mapping workload name to a `Workload`, and is the single source of truth for the on-device layout. A `Workload` carries its per-backend `KernelEntry` table plus optional `flop_fn` / `bytes_fn` callables used to derive throughput for kernels that do not print their own operation counts.

`KernelEntry` field	Meaning
`subdir`	Directory under the device base; `""` means the base itself.
`variants`	Variant name → binary filename.
`default`	Variant used when the task does not name one.
`ldpath`	`LD_LIBRARY_PATH` for GPU dispatches. Default `/vendor/lib64`; `"."` where the kernel ships its own libraries.
`variant_is_flag`	`True` when the variant is selected by a `--variant` flag on a single binary rather than by choosing a different file. True for the DSP saxpy/reduction/convolution entries and for GPU `nbody`.
`defaults`	Flags always applied unless the task overrides them, e.g. `power=lab` on DSP, `perf_mode=sustained` on NPU.
`fixed_size`	Variant → the only `--size` it accepts (§6).

KernelEntry field	Meaning
param_map	Rename a parameter on the way to the CLI, where the kernel spells it differently (§7.2).
threaded_variants	Which variants accept --threads (§5.4). Empty means the whole entry is single-threaded.
totals_with_per_iteration_time	This kernel prints flop=/bytes= covering *all* iterations while its timing field measures *one* (§7.5).

Reading WORKLOADS is the supported way to enumerate what exists:

```
for name, wl in WORKLOADS.items():
    for backend, entry in wl.kernels.items():
        print(name, backend, sorted(entry.variants), "default:", entry.default)
```

6 Kernel and variant catalogue

Generated equivalent: python profiler.py --catalogue. That output is built from the registry at runtime and cannot drift; this table is hand-maintained. Defaults marked below are applied automatically and can be overridden by passing the same keyword to the task. An asterisk marks the default variant.

Workload	Bk	Variants	Applied defaults / notes
matmul	CPU	neon*, eigen	iterations=10; neon is threaded
matmul	GPU	naive*, tiled, fp16, vec4, fp32_3x16u4, fp32_3x16u4_db, fp32_4x16u4, fp32_8x16u4, anysize, fp16_best, best	ldpath="."; the six register-blocked variants are **size=512 only**
matmul	DSP	scalar, hvx*, vtcm	separate binaries, not flag-selected
matmul	NPU	fp16*, int8	perf_mode=sustained, reps=16; size→--n, iterations→--reps
saxpy	CPU	scalar, neon*	iterations=200; both threaded
saxpy	GPU	fp32*	—
saxpy	DSP	scalar, hvx*	one binary, --variant; power=lab, iterations=50
saxpy	NPU	fp16*	perf_mode=sustained, reps=5
reduction	CPU	scalar, neon*	iterations=200; both threaded
reduction	GPU	fp32*	—
reduction	DSP	scalar, hvx*	--variant; power=lab, iterations=50
reduction	NPU	fp16*	perf_mode=sustained, reps=5
convolution	CPU	scalar, neon*, int8	iterations=10; neon and int8 threaded, scalar not
convolution	GPU	fp32*	—
convolution	DSP	scalar, hvx*, int8	--variant; power=lab, iterations=2
convolution	NPU	fp16*	perf_mode=sustained, reps=5
nbody	CPU	neon*	iterations=4; threaded

Workload	Bk	Variants	Applied defaults / notes
nbody	GPU	`naive`, `tiled`*	one binary, `--variant`; `iterations=5`
decode	CPU	`neon`*	—
decode	GPU	`naive`*, `gemv`	`ldpath="."`, `wg-x=64`, `wg-y=8`; `n`→`--ncols`
decode	NPU	`fp16`*	`perf_mode=sustained`, `reps=20`; `n`→`--ncols`
cachelat	CPU	`latency`*	`accesses=40000000`, `pattern=random`
morph	DSP	`morph`*	—
gpu_memlab	GPU	`memlab`*	`mode=coalesce`
custom	all four	—	you supply the binary name (§9)
model	CPU, GPU, NPU	`qnn`*	dispatched through `qnn-net-run` (§10)

The GPU matmul variant trap. `matmul` on the GPU defaults to `naive`, which is roughly 16× slower than `best`. That default is deliberate — it keeps the optimisation ladder visible — but it makes cross-backend comparison meaningless if you forget it. A GPU-versus-NPU matmul run at the default variant measures an unoptimised OpenCL kernel against a tuned tensor engine and tells you nothing about the silicon. Name `variant="best"` for any cross-engine claim.

The register-blocked variants accept exactly one size. `best`, `fp16_best`, `fp32_3x16u4`, `fp32_3x16u4_db`, `fp32_4x16u4` and `fp32_8x16u4` are compiled for $N = 512$ and assert on anything else. The API checks this at construction:

```
ValueError: variant 'best' is compiled for size=512 only, got size=1024.
Use an any-size variant (naive, tiled, vec4, fp16, anysize) to sweep sizes.
```

`anysize` is the register-blocked kernel recompiled to accept arbitrary N; use it when you want the blocked shape *and* a size sweep.

The register-tile ladder. `vec4` → `fp32_3x16u4` → `fp32_4x16u4` → `fp32_8x16u4` sweeps tile shape, which is the knob that actually moved GFLOP/s on this GPU. `fp32_4x16u4` and `fp16_best` share the 4×16 tile, so that pair isolates *precision* alone; `best` is a different (3×16) shape. More variants exist unregistered under `kernels/matmul/gpu/`.

7 Parameter reference

7.1 How keywords become flags

Python keyword	Becomes	Rule
`size=1024`	`--size 1024`	scalar values are stringified
`perf_mode="sustained"`	`--perf-mode sustained`	underscores become hyphens
`H=256`	`--H 256`	case is preserved
`validate_only=True`	`--validate-only`	`True` emits a bare flag with no value

Python keyword	Becomes	Rule
`iterations=None`	(dropped)	`None` and `False` are omitted

Values are shell-quoted with `shlex.quote`, so strings with spaces are safe. Entry defaults are applied first and task parameters overwrite them. Ordering within the command is defaults-then-overrides, which is why `--perf-mode sustained` appears before `--cin` in the NPU examples.

7.2 Parameter renaming: `param_map`

Three registry entries rename a parameter on its way to the CLI, because the kernel spells it differently:

Entry	Renames
matmul / NPU	`size` → `--n`, `iterations` → `--reps`
decode / GPU	`n` → `--ncols`
decode / NPU	`n` → `--ncols`

This exists to make the 4×4 matrix uniform. Before it, NPU matmul was the one cell where `iterations=N` hard-failed, so a loop over all four backends died on one engine; and `size` had to be passed twice (once as `n` for the kernel, once as `size` for the FLOP model). Now `matmul(Backend.NPU, size=1024)` sends `--n 1024` *and* computes $2 \cdot 1024^3$ FLOP, because `flop_fn` reads the task's original parameters while the rename happens only on the way to the shell.

The corresponding hazard: a rename that is missing or wrong does not error. Kernels ignore unknown flags silently, so the dispatch succeeds and quietly measures the kernel's *default* shape instead of yours.

7.3 Parameters accepted by each kernel

Workload	Bk	Accepted parameters
matmul	CPU	`size`, `iterations`, `seed`, `validate_only`
matmul	GPU	`size`, `seed`, `wg_x`, `wg_y`, `wg_z`; blocked variants also take `inner_reps`
matmul	DSP	`size`, `seed`
matmul	NPU	`size` (renamed to `--n`), `iterations` (renamed to `--reps`), `perf_mode`, `seed`
saxpy	CPU	`size`, `alpha`, `iterations`, `seed`, `validate_only`
saxpy	GPU	`size`, `alpha`, `seed`, `wg_x`, `wg_y`, `wg_z`
saxpy	DSP	`size`, `alpha`, `iterations`, `power`, `seed`, `validate_only`
saxpy	NPU	`size`, `alpha`, `reps`, `perf_mode`, `seed`
reduction	CPU	`size`, `iterations`, `seed`, `validate_only`
reduction	GPU	`size`, `seed`, `wg_x`, `wg_y`, `wg_z`
reduction	DSP	`size`, `iterations`, `power`, `seed`, `validate_only`
reduction	NPU	`size`, `reps`, `perf_mode`, `seed`
convolution	CPU	`H`, `W`, `cin`, `cout`, `k`, `iterations`, `seed`, `validate_only`
convolution	GPU	`H`, `W`, `cin`, `cout`, `k`, `seed`, `wg_x`
convolution	DSP	`H`, `W`, `cin`, `cout`, `k`, `iterations`, `power`, `seed`

Workload	Bk	Accepted parameters
convolution	NPU	`H`, `W`, `cin`, `cout`, `k`, `reps`, `perf_mode`
nbody	CPU	`bodies`, `iterations`, `seed`
nbody	GPU	`bodies`, `iterations`, `seed`
decode	CPU	`tokens`, `k`, `n`, `iterations`
decode	GPU	`tokens`, `k`, `n` (→`--ncols`), `wg_x`, `wg_y`
decode	NPU	`tokens`, `k`, `n` (→`--ncols`), `reps`, `perf_mode`
cachelat	CPU	`kb`, `accesses`, `pattern`, `stride`, `smallpages`, `hugepages` (both bare flags)
morph	DSP	`op` (erode/dilate), `impl` (student/qhl), `damage` (salt/pepper), `k`, `input` (P5 PGM, default `dog.pgm`), `W`, `H` (synthetic image only), `iterations`, `write_pgm`
gpu_memlab	GPU	`mode` (coalesce/divergence/spmv/densemv), `stride` (in 4-byte floats), `kb` (buffer size in KiB), `active`, `density`, `rows`, `cols`, `local` (workgroup size), `reps`

Do not pass `variant` or `power` as ordinary parameters on the DSP. `variant` is set for you from `Task.variant` on the entries that are flag-selected, and `power` already defaults to `lab`. Override `power` only when you specifically want the baseline DCVS behaviour: `power="baseline"`.
`cachelat`: page size and run length. This board runs transparent huge pages in *always* mode, so the buffer already gets 2 MB pages with no flag, and `-p hugepages=true` changes nothing. `-p smallpages=true` forces 4 KB pages; compare it against a run without it to see what the TLB costs. The default `accesses=40000000` finishes in about 40 ms when the buffer fits in L1, which is too short to trust. Set `-p accesses=` yourself (200000000 gives about 180 ms on a Prime core).
`morph`: residual and images. Every run reports `residual=`. That is the number of pixels where your output differs from the same filter applied to the undamaged photo, i.e. damage that survived. Lower is better. Add `-p write_pgm=true` to any run and the profiler saves before and after pictures to `morph_images/` in the container. To open them on your laptop, run `docker cp lab3:/workspace/morph_images .` in a terminal on your machine.
`gpu_memlab`: modes, units and power runs. One binary, four tests, picked with `mode`. `coalesce` (the default) reads with `stride` floats between neighbouring work-items and reports GB/s. `divergence` lets `active` lanes out of every 64 do the work and reports GFLOP/s. `spmv` and `densemv` multiply a `rows` × `cols` matrix by a vector, sparse at the given `density` or dense. Units: `stride` counts 4-byte floats, not bytes, and `kb` is in KiB. Keep `-p kb=65536` (64 MiB); `-p kb=64` is legal but runs a 64 KiB buffer that fits in cache. Times are device-side OpenCL events, the minimum over `reps` launches, so short runs are fine for `p.run()`. For `p.power()` use a large `reps` (e.g. 5000): the power loop repeats the whole command, setup and buffer upload included, and with few reps the GPU is idle for much of the window.

7.4 Derived FLOP and byte counts follow *your* parameters

Where a kernel prints `flop=` or `bytes=`, the printed value always wins. Where it does not, the workload's `flop_fn` / `bytes_fn` computes one from the task's parameters:

Workload	Model
matmul	$2n^3$ from `size` (default 512)
convolution	$2 \cdot H \cdot W \cdot c_{out} \cdot k^2 \cdot c_{in}$ from `H`, `W`, `cin`, `cout`, `k` (defaults $256, 256, 8, 8, 3$)

Workload	Model
nbody	$20n^2$ from `bodies` (default 4096), with n **rounded down to a multiple of 64** — both kernels do that internally, so the model mirrors it rather than reporting a rate computed from a body count nobody simulated
decode	FLOP $2 \cdot S \cdot k \cdot n$ from `tokens`, `k`, `n`; bytes $= 4kn$, *weight traffic only.* That accounting is right while $S \ll k$, which is the regime the workload exists to show, and stops being right for large S
saxpy, reduction	no model — these kernels print their own `flop=` and `bytes=`

The consequence is worth stating plainly. If you change the shape on the device without changing the corresponding Python parameter, the timing stays correct and the derived throughput goes silently wrong. The model has no way to see the device.

7.5 Iteration accounting, and why it is not obvious

Kernels repeat their work internally, and they disagree about whether the time they print covers one repetition or all of them. Getting this wrong misreports throughput by exactly the iteration count, so the API corrects for it in two opposite directions.

Situation	What the API does
Time is a total, work model is per-iteration. The CPU kernels report `wall_us` across every repetition, while `flop_fn` describes one dispatch.	Scale the work *up* by the repeat count. Without this, CPU NEON matmul at `iterations=20` reported 1.37 GFLOP/s instead of ≈29.
Work is a total, time is per-iteration. The DSP saxpy, reduction and convolution kernels *print* `flop=`/`bytes=` summed across every iteration, while `dsp_us` is measured inside the DSP and covers one.	Scale the work *down*, flagged by `totals_with_per_iteration_time` on those entries. Without this every DSP figure inflates by the iteration count — 50× on saxpy/reduction, 2× on convolution, at the registry defaults.

Only `wall_us` is a total. `execute_us_mean`, `dsp_us`, `mean_us` and `duration_ns` are already per-dispatch — a mean, or one timed region — and must never be divided by the repeat count. Doing so reported an NPU convolution as 32 μs/iter when the real per-inference time was 172.

`Result.iterations` exposes the repeat count the API used, and `Result.compute_us_per_iter` the per-dispatch latency. `summary()` prints `N us/iter xM` rather than a bare time

whenever $M > 1$, so the total stays recoverable and nobody reads a ten-iteration convolution as one slow one.

8 What the fields mean, per backend

8.1 Selection of `compute_us`

The parser takes the first field present, in this order. `Result.compute_key` records which one won.

Field	Backend	What it measures
`execute_us_mean`	NPU	Mean of `reps` graph executions, excluding graph finalisation.
`dsp_us`	DSP	Time measured on the Hexagon itself, excluding FastRPC marshalling. Per iteration.
`mean_us`	varies	Mean over a kernel's own measured iterations.
`duration_ns`	GPU	OpenCL profiling event, kernel only, excluding host transfers. Divided by 1000 into μs.
`wall_us`	CPU	Host-measured wall clock around the timed region. **The only total.**

Every one of these is *compute only*. Comparing `compute_us` across engines compares the engines; it does not tell you which is fastest end-to-end. A kernel that prints none of them is treated as a contract violation and reported as a failure, not as a success with a blank time — see §9.

8.2 Transport and overhead fields

These live in `Result.fields` and are never folded into `compute_us`:

Field	Backend	Meaning
`transfer_ns`	GPU	Host → device input copy. On SAXPY this runs from 0.76× to 3.6× the kernel time.
`readback_ns`	GPU	Device → host result copy. Collapses to a few microseconds when the output is a scalar (reduction).
`cl_start_ns`, `cl_end_ns`	GPU	Raw OpenCL event timestamps.
`useful_gbps`	GPU	Some kernels compute their own rate because only they know what counts as *useful*. `gpu_memlab` is the case in point: the bytes the memory system moves can differ from the bytes the kernel actually uses, and the host cannot see that from outside. Picked up into `Result.gbps` when nothing else supplies it.
`ns_per_access`	CPU	`cachelat`'s headline number, and the ruler the cache hierarchy is measured with. Promoted into `summary()`.
`finalize_us`	NPU	QNN graph compilation. Roughly 10 ms for convolution, 4.4 ms for reduction, and **50–110 seconds** for SAXPY, which has no native primitive.
`wall_us`	DSP, NPU	Includes FastRPC / QNN transport, so the gap against `compute_us` is the marshalling tax (up to two orders of magnitude on DSP streaming kernels).

Field	Backend	Meaning
`execute_us_min`, `_max`, `_cov_pct`	NPU	Spread across `reps`; the coefficient of variation is the cheapest stability check you have.
`host_wall_us`	all	Added by the profiler, not the kernel: the host-observed round trip including `adb` latency. Always present. For `model` tasks it is the *only* timing available.
`fingerprint`	all	Numerical checksum. Compare it against a known-good run before trusting a timing.

8.3 What counters mean on a non-CPU backend

The `simpleperf` prefix is applied on *every* backend, not just the CPU, and the dispatch succeeds. **What you get back is the ARM process that drives the accelerator, not the accelerator.** A DSP run with `counters="basic"` reports the IPC of the CPU core executing the FastRPC client while the Hexagon does the arithmetic. That is a genuinely useful measurement — it is how you see what the host is paying to keep an engine fed — but it is not the engine's IPC, and there is no way to get the engine's IPC from here. GPU, DSP and NPU hardware counters are not exposed on this board.

8.4 Reading a result correctly

```
r = p.run(saxpy(Backend.NPU, size=65536, timeout=300))

r.ok                        # did it run at all
r.fingerprint               # is the answer right
r.compute_us                # engine-only time
r.compute_key               # ...measured how? ('execute_us_mean' here)
r.iterations                # ...covering how many repetitions?
r.fields["finalize_us"]     # the compile cost you would pay in production
r.wall_us                   # what an end-to-end user would actually feel
r.raw                       # everything, unmodified, when the above disagree
```

9 Your own kernels: `custom`

`custom()` dispatches a binary you wrote, so adding a kernel never means editing `profiler.py`.

```
from profiler import Profiler, Backend, custom

p = Profiler()
r = p.run(custom(Backend.CPU, "my_kernel", size=1048576, core="prime"))
```

`binary` is a filename under `/data/local/tmp/profiler-kernels/custom/`; the backend chooses the environment (library paths, affinity) exactly as for a registered kernel. Build and push with `python scripts/new_kernel.py --push <name>`.

The output contract

Your kernel must print **one timing field**, chosen from `mean_us`, `wall_us`, `duration_ns`, `dsp_us` or `execute_us_mean`. This is required: a run with no time is reported as a failure, with the contract quoted in `Result.error`. Leaving it as a success produced a blank summary line and no error, which is the least helpful thing a first custom kernel can do.

Optionally print `flop:`, `bytes:`, `fingerprint:` and `status:`. Only `OK`, `SUCCESS`, `PASS`, `PASSED` and `DONE` count as success; anything else marks the result failed. Most shipped kernels print no status at all and rely on the exit code.

Either output style is parsed: `key=value` tokens anywhere on a line, or a `key: value` line on its own.

Two limitations

- `custom()` accepts `name`, `timeout`, `depends_on` and `core`, but **not** `threads`, `counters` or `per_core`. To use those, construct the `Task` directly with `binary=`.
- `custom` resolves binaries under `custom/` only. A registered workload's binary cannot be driven through it — which is why `cachelat` is in the registry rather than left to `custom`: it needs counters and lives in `cachelat/`.

10 Converted networks: `model`

Networks are not kernels. They are DLC files executed by QAIRT's own `qnn-net-run`, and the engine is chosen by which QNN backend library is loaded — `libQnnCpu.so`, `libQnnGpu.so` or `libQnnHtp.so`. One DLC runs on any of the three, so **the engine is the variable and the model is fixed**, which is the whole point of the case study.

```
from profiler import Profiler, Backend, model

p = Profiler()
r = p.run(model(Backend.NPU, "minilm", reps=10))
print(r.wall_us)                        # the only timing qnn-net-run gives

for b in (Backend.CPU, Backend.GPU, Backend.NPU):
    print(p.time_model(b, "mobilenetv2", reps=100))
```

The DSP is deliberately absent: QAIRT ships a QNN DSP backend, but provisioning does not push it, so listing it would advertise a dispatch that cannot run.

10.1 The model registry

`MODELS` maps a short name to its directory, DLC, input list, MAC count and kind. All eleven live under `/data/local/tmp`, staged separately from the kernel tree.

Name	Kind	MACs	Notes
`mobilenetv2`	cnn	301 M	depthwise-separable, linear topology
`mnv2_s1`	cnn	120.8 M	stage 1 of 3 (40% of MACs)
`mnv2_s2`	cnn	104.6 M	stage 2 of 3 (35%)
`mnv2_s3`	cnn	75.3 M	stage 3 of 3 (25%)
`minilm`	encoder	15 M	6-layer BERT encoder, seq=128
`distilgpt2`	decoder	88.4 M	6-layer decoder PREFILL, seq=128, no KV cache
`qwen05_dec32`	decoder	494 M	Qwen2.5-0.5B decode, 2.52 GB DLC
`llama1b_dec32`	decoder	1236 M	Llama-3.2-1B decode, 6.00 GB DLC
`sd_text_encoder`	clip	6.7 G	CLIP text encoder, once per image
`sd_unet`	unet	68 G	distilled SD UNet, **once per denoising step**
`sd_vae_decoder`	vae	1.3 T	VAE decoder; the most engine-sensitive stage on this board (8× spread)

The three `mnv2_s*` stages exist so a network can be cut and the pieces placed on different engines. The diffusion trio is the pipeline where that actually pays: its stage boundary is a 64 KB latent, cheap enough for a genuine multi-engine pipeline, which the LLM's 4.7 MB KV handoff never allowed.

The decode models are fp32 and that is not a choice. The CPU backend refuses fp16 DLCs (Discovery 71), so the DLCs are large and decode is weight-bound. **The diffusion text encoder takes token *embeddings*, not ids**: the converter folds integer inputs away (Discovery 77), so the 77-row lookup happens on the host and only the twelve transformer layers run on device.

10.2 `model(backend, name, **params)`

Argument	Meaning
`backend`	`Backend.CPU`, `.GPU` or `.NPU`. Anything else raises.
`name`	A key of `MODELS`. Unknown names raise, listing the known ones.
`reps`	`--num_inferences`: how many inferences one dispatch performs. Default 1.
`output_dir`	Where `qnn-net-run` writes outputs on device. Defaults to `run_<model>_<backend>`, and is deleted before each run.
`name=`	Task name, defaulting to `model.<name>.<backend>`.

A `model` result has **no `compute_us`** — `qnn-net-run` prints no timing field the parser recognises. Its `wall_us` is filled in by the profiler from the host round trip, and `ok` additionally requires the string *"Finished Executing"* in the output. A model that was never staged fails with a message naming the command that stages it, rather than a bare `cd: No such file or directory`.

10.3 `time_model(backend, name, reps=100)`

Graph preparation is paid once per dispatch and differs enormously by backend — 0.16 s on CPU, 0.52 s on GPU, 0.59 s on HTP for MobileNetV2 — so a single timed run measures mostly compilation. `time_model` runs the model twice, once with `reps=1` and once with `reps+1`, and subtracts. The subtraction cancels preparation and the constant `adb` round trip.

```
{'model': 'mobilenetv2', 'backend': 'npu', 'ok': True,
 'per_inference_ms': 0.812, 'prepare_plus_one_ms': 604.3,
 'inferences_per_s': 1231.5, 'reps': 100, 'macs': 301000000.0}
```

It refuses rather than returning a negative number. The subtraction only cancels preparation if preparation costs the same in both runs, and it often does not — a cold DLC read from storage can dominate the first run, after which the `reps+1` run comes back *faster* in wall time and the per-inference figure goes negative. Reporting −0.75 ms as a measurement is worse than reporting nothing, so the call returns `ok: False` with an error saying to raise `reps` until the per-inference signal outweighs the one-off load.

11 Power estimation: `power()`

```
p.power(matmul(Backend.CPU, size=1024))
# {'available': True, 'power_w': 6.59, 'energy_j': 230.6, 'window_s': 35,
#  'engine': 'cpu', 'boundary': '12V supply input',
#  'scope': 'whole_board_not_per_engine', 'board_match': 'calibration_board'}
```

11.1 Three things this is not

Each matters more than the number itself.

Not a measurement.
It is a calibrated *model* of 12 V supply-input power, fitted against a bench supply. This board has no readable PMIC rail — the battery node is a `Debug_Board` dummy (Discovery 57) — so there is nothing to read. The returned `quality` is `modeled_ballpark` for that reason.

Not per-engine.
It predicts what the *whole board* draws. "GPU watts" is not a thing this can produce.

Not per-dispatch.
The model is validated only over intervals of at least `minimum_validated_interval_s` (30 s), so the task is looped to fill `seconds` and the answer describes that window. A single 200 μs dispatch has no modeled power. Asking for fewer seconds than the artifact's validated interval returns `available: False`.

11.2 Arguments

Argument	Meaning
`task`	The `Task` to loop. One task, one engine.
`seconds`	Window length, default 35. Must be $\geq$ the artifact's validated interval (30 s).
`artifact`	Path to a power-model JSON. Default: the best available for the attached board — the SM8850 CPU/GPU artifact first, then the SM8750 ones, so a Hexagon calibration appears automatically the moment one lands.

11.3 What each engine returns

Engine	Status
CPU	Fitted from measurements.
GPU	Fitted from measurements.
NPU	*Declared*, not fitted: 5 W fp16, 4 W int8, hand-entered from operator readings. Returned with `validated: False` so a reader can tell a fitted number from a typed-in one, and returned *immediately* without running a window — running a 35 s NPU window and then ignoring its features would be theatre, and it is exactly the run that drives the die to 98 °C (Discovery 73).
DSP	`available: False`, with a reason. An unsigned PD cannot cast the HAP power vote the calibration needs. **Reporting unavailable is the correct answer, not a fallback number** — see ADRs 0019, 0020 and 0022.

11.4 Board transfer

The model's coefficients were fitted on one specific board. `board_match` says which case you are in:

- `calibration_board` — fitted on this exact unit.

- `transferred_eas_verified` — fitted elsewhere, and this board's kernel energy model was checked against the calibration board's before the coefficients were applied. More uncertainty in the absolute watts; *none* in a comparison between two runs on this board.

A serial mismatch with no EAS verification blocks the estimate outright. Two same-model HDK8850 units differed by 4.44%/4.72% in their vendor EAS tables, so the silicon genuinely is not identical.

11.5 Why there is no multi-engine power call

The model's concurrent-path terms are not summable (`concurrent_path_terms_summable: false`), so there is no call that gives the power of a multi-engine pipeline. Measure each stage and add the *energies*.

11.6 Operational notes

The call takes about a minute and prints a progress line to stderr, so it does not look hung. The device-side loop is given `seconds + 25` so it cannot end mid-window, which means it outlives the measurement; `power()` reaps it afterwards, because otherwise a second `power()` call lands while the first loop is still running and is refused with *"compute clients are already running"*.

`reason` explains why there is no number; `warnings` says what to distrust about one that is present. Read both.

12 Command-line interface

The module is runnable directly, for single points, for the catalogues, and for the demo tour.

```
python profiler.py --demo
python profiler.py --catalogue
python profiler.py --list-counters

python profiler.py matmul npu
python profiler.py matmul gpu --variant best -p size=512
python profiler.py reduction cpu --core prime -p size=4194304
python profiler.py convolution cpu --core gold --threads 6 --counters cache --per-core
python profiler.py convolution npu -p cin=64 -p cout=64 -v
python profiler.py cachelat cpu -p kb=256 -p pattern=random --repeat 5
python profiler.py custom cpu --binary my_kernel -p size=1048576
python profiler.py saxpy dsp --serial 5cabf4fd
```

Flag	Meaning
`--demo`	Run the live tour. Also the default when no workload is given.
`--catalogue`	Print every workload, backend, variant and applied default, generated from the registry.
`--list-counters`	Print the presets, the 92 verified events by group, and the 43 blocked ones with reasons. Exits without touching the device.
`--serial SERIAL`	Target one board when several are attached.
`-v`, `--verbose`	Print each generated device command.
`--variant NAME`	Kernel variant.
`--core SPEC`	Affinity: a core (`7`), a mask (`0xc0`), or `prime`/`gold`/`all`.
`--threads N`	Worker threads (threaded variants only).
`--counters SPEC`	A preset name or a comma-separated event list.
`--per-core`	Break counter totals down per CPU.

Flag	Meaning
`--counter-cpus SPEC`	Count per-CPU system-wide on these CPUs (e.g. `0-5`) instead of per-process.
`-p KEY=VAL`	A kernel parameter; repeatable. Values are coerced: integers, then floats, then `true`/`false` into bare flags, else left as a string.
`--repeat N`	Run the whole dispatch N times and report mean, range and coefficient of variation.
`--binary NAME`	With workload `custom`: the binary under the device `custom/` directory. Rejected with any other workload.

12.1 `--repeat`: process repeats, not kernel iterations

`--repeat N` runs N separate *processes*. That is the point: process-level repeats re-pay startup and let the governor settle differently each time, so the spread they expose is the spread a real caller would see. A kernel's own `--iterations` cannot show you that — it amortises it away.

The summary picks whichever headline metric the workload actually reports (GFLOP/s, else GB/s, else `wall_us`):

```
  1  matmul.cpu    105234.1 us/iter x10    29.3 GFLOP/s  fp=1.3587e+03
  2  matmul.cpu    106881.7 us/iter x10    28.9 GFLOP/s  fp=1.3587e+03
  3  matmul.cpu    105902.4 us/iter x10    29.1 GFLOP/s  fp=1.3587e+03

n=3  GFLOP/s: mean 29.1  range 28.9-29.3  CoV 0.7%
```

Failed runs are excluded from the statistics and counted in a note. If the runs produce more than one distinct fingerprint, a warning says so — they did not all compute the same answer, and the timing spread is the least of your problems.

One run is not a measurement. `--repeat` exists to make that easy to act on.

13 Recipes

13.1 One roofline point on every engine

```python
from profiler import Profiler, Backend, convolution

p = Profiler()
for r in p.run_serial([convolution(b, cin=8, cout=8) for b in Backend]):
    if r.ok:
        print(f"{r.task.backend}  {r.compute_us:10.1f} us  {r.gflops:8.1f} GFLOP/s")
    else:
        print(f"{r.task.backend}  FAILED: {r.error}")
```

13.2 A parameter sweep written to CSV

```python
import csv
from profiler import Profiler, Backend, matmul

p = Profiler()
with open("matmul_gpu.csv", "w", newline="") as fh:
    w = csv.writer(fh)
    w.writerow(["size", "variant", "compute_us", "gflops", "fingerprint"])
    for size in (128, 256, 512, 1024):
        for variant in ("naive", "tiled", "anysize"):   # any-size variants only
            r = p.run(matmul(Backend.GPU, size=size, variant=variant))
            if r.ok:
```

```
                w.writerow([size, variant, r.compute_us, r.gflops, r.fingerprint])
```

13.3 Contention: the same batch, both ways

```
batch = [matmul(Backend.CPU, size=512), matmul(Backend.GPU, size=512, variant="best")]

clean     = p.run(batch, parallel=False)
contended = p.run(batch, parallel=True)

for a, b in zip(clean, contended):
    print(f"{a.task.name:20s} {a.compute_us:9.1f} -> {b.compute_us:9.1f} us "
          f"({b.compute_us / a.compute_us:.2f}x)")
```

13.4 CPU cluster and core study, with counters

```
for spec in ("prime", "gold", 7, 0):
    r = p.run(reduction(Backend.CPU, size=4 << 20, core=spec,
                        counters="basic", name=f"core_{spec}"))
    d = r.derived
    print(f"{str(spec):6s} {r.compute_us:9.1f} us  "
          f"{d.get('effective_ghz', 0):.2f} GHz  "
          f"{d.get('clock_of_max_pct', 0):5.1f}% of max  IPC={d.get('ipc', 0):.2f}")
```

Name the cluster. Under the default mask `clock_of_max_pct` is absent, because cores 4–7 straddle the boundary (§5.3).

13.5 Thread scaling, done so the answer means something

```
for n in (1, 2, 3, 4, 5, 6):
    r = p.run(convolution(Backend.CPU, cin=32, cout=32, iterations=20,
                          threads=n, core="gold", variant="neon",
                          counters="cache", name=f"conv_{n}t"))
    print(f"{n} threads  {r.compute_us_per_iter:9.1f} us/iter  "
          f"IPC={r.derived.get('ipc', 0):.2f}  "
          f"L2 miss={r.derived.get('l2d_miss_pct', 0):.2f}%")
```

Three things make this measurement honest rather than misleading: `iterations=20` so thread spawn is amortised (§5.4); `core="gold"` so six threads have six cores; and `variant="neon"` because `scalar` would raise.

13.6 Scalar versus vectorised, on one engine

```
for variant in ("scalar", "neon"):
    r = p.run(saxpy(Backend.CPU, size=1 << 22, variant=variant, core="prime"))
    print(f"{variant:7s} {r.gbps:6.1f} GB/s")
```

13.7 A pipeline expressed as a DAG

```
results = p.run_dag([
    reduction(Backend.CPU, size=1 << 20, name="stage1"),
    convolution(Backend.GPU, cin=16, cout=16, name="stage2a", depends_on=["stage1"]),
    convolution(Backend.NPU, cin=16, cout=16, name="stage2b", depends_on=["stage1"]),
```

```
    matmul(Backend.CPU, size=512, name="stage3", depends_on=["stage2a", "stage2b"]),
])
for name in ("stage1", "stage2a", "stage2b", "stage3"):
    print(results[name].summary())
```

13.8 Comparing engines on one network

```
for b in (Backend.CPU, Backend.GPU, Backend.NPU):
    t = p.time_model(b, "mobilenetv2", reps=100)
    if t["ok"]:
        print(f"{b}  {t['per_inference_ms']:7.3f} ms  "
              f"{t['inferences_per_s']:8.1f} inf/s  "
              f"(prepare {t['prepare_plus_one_ms']:.0f} ms)")
    else:
        print(f"{b}  {t['error']}")
```

13.9 Checking what will run, without a board

```
from profiler import build_command, DEVICE_BASE, WORKLOADS, Backend, matmul

for variant in WORKLOADS["matmul"].kernels[Backend.GPU].variants:
    print(f"{variant:16s} {build_command(matmul(Backend.GPU, variant=variant), DEVICE_BASE)}")
```

13.10 A guarded sweep that reports its own failures

```
results, failures = [], []
for size in (256, 512, 1024, 2048):
    r = p.run(matmul(Backend.DSP, size=size, timeout=600))
    (results if r.ok else failures).append(r)

print(f"{len(results)} ok, {len(failures)} failed")
for r in failures:
    print(f"  {r.task.name} size={r.task.params['size']}: {r.error}")

fps = {round(r.fingerprint, 6) for r in results if r.fingerprint is not None}
if len(fps) > 1:
    print(f"WARNING: {len(fps)} distinct fingerprints -- not all the same answer")
```

14 Extending the wrapper

14.1 Running a kernel without touching this file

Use `custom()` (§9). Nothing needs registering. This is the right answer for a one-off, for coursework, and for anything you are still iterating on.

14.2 Adding a variant to an existing kernel

Push the binary, then add one entry to the `variants` dict:

```
Backend.GPU: KernelEntry("gpu", {"naive": "matmul_opencl_fp32",
                                 "mine":  "matmul_opencl_fp32_mine"},
                         "naive", ldpath="."),
```

If it is compiled for one shape only, add it to `fixed_size` so the guard fires on the host. If it accepts `--threads`, add it to `threaded_variants`.

14.3 Adding a backend to an existing workload

Add a `KernelEntry` under the new `Backend` key. The environment (working directory, library paths, affinity, counter prefix) is applied by backend, so nothing else changes. If the kernel spells a parameter differently, add a `param_map` entry rather than making callers remember it.

14.4 Adding a whole workload

Add a `Workload` to `WORKLOADS` with its `kernels` table, and a `flop_fn` / `bytes_fn` if the kernels do not print their own counts. Then add a module-level constructor:

```
def myworkload(backend: Backend, **params) -> Task:
    """One line saying what the workload is for and where it sits on the roofline."""
    return _task("myworkload", backend, params)
```

Add the name to `__all__`, or `from profiler import *` will silently not bind it.

14.5 Adding a new timing field

Append it to `_COMPUTE_FIELDS` in priority order. If the new field is a *total* across repetitions rather than a per-dispatch figure, also add it to `_TOTAL_COMPUTE_FIELDS` — otherwise throughput is wrong by exactly the iteration count (§7.5).

14.6 Adding a derived counter metric

Append a tuple to `_DERIVED`: `(name, numerator candidates, denominator candidates, scale)`. The first candidate present wins, and the metric is computed only when both sides exist — so a metric that needs events the caller did not select simply does not appear, rather than appearing as zero.

15 Semantics, caveats, and measurement validity

15.1 The caveats that change conclusions

NPU and DSP do not run in parallel.
They share the Hexagon cDSP. Parallel submission looks right from the host and serialises in hardware. §4.4.

The DSP cannot pin its own clock.
HAP power voting is refused from the unsigned PD we run in (`0x80000414`), with no testsig installed. Compute over FastRPC works fine; only the power vote does not. **Consequence: DSP numbers are unpinned and vary more than the other backends.** Two identical DSP runs differing by 50% is normal, not a bug. Use `--repeat` and quote a spread.

A busy NPU makes the DSP faster.
Same shared DVFS domain. A DSP kernel measured beside an NPU peer can be 1.59× faster than the same kernel alone. Every DSP figure here is the idle-Hexagon case.

The default affinity mask spans two clusters.
`core=None` is cores 4–7: two Gold, two Prime. Name the cluster for anything you will quote. §5.3.

Defaults are not the fastest variant.
GPU matmul defaults to `naive`, about 16× off `best`. Deliberate, and fatal to a cross-engine comparison if forgotten.

NPU SAXPY needs a long timeout.
QNN has no native SAXPY operator, so the graph decomposes into thousands of tiles and `finalize_us` runs 50–110 seconds. The default 180 s timeout is not always enough; pass `timeout=300`. The wait *is* the finding.

GPU counters are not available.
Adreno perfcounters are blocked on this build. Counters on a GPU task measure the host process. §8.3.

Never run `adb root`.
It restarts `adbd` in a domain that cannot reach the OpenCL ICD, so every GPU dispatch fails with `clGetPlatformIDs` = −1001. Counters use `su 0`, which is a different and safe mechanism. §4.3.

The NPU can destabilise the board in still air.
A sustained fp16 graph reached 98 °C with mitigation engaged, at only 4–5 W, and the board dropped off USB. Power density, not total power. Do not leave a sustained NPU sweep running unattended without airflow.

Thermals are tighter than the previous board.
This HDK cannot hold its top GPU or CPU operating points under load in still air. Several published calibration setpoints sit below the top of the ladder for that reason.

15.2 Checking a number before you quote it

1. `r.ok` is `True`.
2. `r.fingerprint` matches a known-good run. A fast wrong answer is still wrong.
3. `r.compute_key` is the field you think it is, and `r.iterations` is what you asked for.
4. `r.below_max_clock` is `False`, or you know why it is not.
5. `r.multiplexed` is `False` if you are reading counter values.
6. You ran it more than once (`--repeat`) and the coefficient of variation is small enough to support the claim.
7. The comparison is like-for-like: same variant class, same sizes, same serial-versus-parallel mode.

16 Troubleshooting

Symptom	Cause and fix
`ValueError: unknown workload`	Typo, or a workload that does not exist. The message lists the known ones.
`ValueError: X has no Y kernel`	That workload does not run on that engine — `nbody` has no NPU, `cachelat` is CPU-only. The message lists the backends it does have.

Symptom	Cause and fix
`ValueError: ... has no variant`	Typo. The message lists every valid variant and often a "did you mean".
`ValueError: compiled for size=512 only`	A register-blocked GPU variant. Use `anysize`, `naive`, `tiled`, `vec4` or `fp16` to sweep sizes. §6.
`ValueError: ... does not accept --threads`	That variant is single-threaded; the message names the threaded ones. §5.4.
`ValueError: counter ... reads a constant 0`	The event is not implemented on this silicon. The message names the substitute. §5.5.
Result `ok=False`, error mentions `No such file`	The binary was never pushed. Re-run `provision_device.py`, then `adb shell ls` the directory to confirm — *"1 file pushed"* is not evidence.
Every counter run fails, no counters at all	`su 0` is unavailable, or you are on a build where the shell domain cannot open the PMU and `su` is absent. Everything except counters still works.
Exit code 124 in `error`	Timeout. Raise `timeout=`; for NPU SAXPY use 300 s or more.
Blank summary, `ok=False`, "printed no timing field"	A custom kernel that does not meet the output contract. §9.
Throughput looks wrong by exactly 10× or 50×	An iteration accounting mismatch. Check `r.compute_key` and `r.iterations`. §7.5.
GFLOP/s right, timing right, but for the wrong shape	A parameter the kernel ignored. Turn on `verbose=True` and read the command.
GPU dispatches all fail with `-1001`	Somebody ran `adb root`. `adb unroot` and reconnect.
`power()` returns `available: False` for the DSP	Working as designed — the DSP is not calibrated and will not be guessed at. §11.
`power()` says "compute clients are already running"	A previous `power()` loop has not been reaped. Wait 25 s, or `adb shell pkill -f profiler_power_loop`.
`time_model` returns negative per-inference time	Load noise exceeded the signal. Raise `reps`. §5.7.
Results differ between two identical DSP runs	Expected: the DSP clock is unpinned. §15.1.
`adb` dies mid-sweep	Intermittent on this board. The provisioner is idempotent and sweeps are restartable; re-run.

A Counter catalogue

Generated equivalent: `python profiler.py --list-counters`. 92 events are verified on this silicon, across ten groups. Each was run against a memory-heavy workload on both clusters and returned a non-zero count. **Note this checks that a counter counts, not that its value is correct.**

Group	N	What it covers
`core`	3	`cpu-cycles`, `instructions`, `bus-cycles`.
`branch`	9	Retired and speculative branches, predictor hits and misses, split by direct / indirect / return.
`cache`	20	L1D, L1I and L2 accesses, refills, reads, writes, writebacks, allocations, long-latency misses, and L1D refills split by inner (on-cluster) vs outer (off-cluster).

Group	N	What it covers
`prefetch`	7	Accesses generated by the *hardware* prefetcher (`*-hwprf`) versus by software `PRFM` instructions (`*-prfm`). A sequential stream drives `hwprf` hard; a pointer chase does not, and that contrast is the experiment.
`tlb`	18	L1 dTLB/iTLB, L2 unified TLB, refills, and page-table walks including walk depth (`*-step`) and whether the walk resolved to a large or a small page.
`stalls`	9	Frontend and backend stalls in both cycles and issue slots, plus memory-attributed backend stalls. Slots are finer than cycles.
`mix`	8	Retired vs speculative instructions and micro-ops, loads, stores, integer data-processing, and FP/SIMD. **NEON counts under `raw-vfp-spec`**, not `raw-ase-inst-spec`, which reads zero.
`memory`	4	Memory accesses (total, read, write) and bus/interconnect accesses.
`exceptions`	2	Exceptions taken and returned.
`aliases`	12	simpleperf's friendlier `hw-cache` names for events already listed above (`L1-dcache-loads`, `dTLB-load-misses`, and so on). Same silicon events.

A.1 Derived metrics

Computed automatically from whichever events are present. A metric whose numerator or denominator is missing simply does not appear.

Metric	Definition
`ipc`, `cpi`	instructions / cycles, and its reciprocal
`branch_miss_pct`	mispredicts / branches
`l1d_miss_pct`, `l1i_miss_pct`, `l2d_miss_pct`	refills / accesses, per level
`l1dtlb_miss_pct`, `l2dtlb_miss_pct`	TLB refills / TLB accesses
`pagewalks_per_kinst`	data page-table walks per thousand instructions
`frontend_stall_pct`, `backend_stall_pct`	stall cycles / total cycles
`mem_stall_pct`	memory-attributed backend stalls / cycles
`slot_backend_pct`	backend-lost issue slots / total lost slots
`l1d_wb_per_kinst`	L1D writebacks per thousand instructions
`l1d_hwprf_pct`, `l2d_hwprf_pct`	share of cache traffic the *hardware prefetcher* generated rather than the program
`spec_waste_pct`	$1 -$ retired/speculative: the fraction of executed instructions that never retired
`effective_ghz`	measured clock, from cycles over simpleperf's window (§5.6)
`clock_of_max_pct`	effective clock as a percentage of the cluster ceiling
`scaling_cur_freq_ghz`	the governor's *requested* clock, sampled during the run

B Glossary of the numbers this document quotes

Constant	Value and meaning
`DEVICE_BASE`	`/data/local/tmp/profiler-kernels`
`MODEL_DIR`	`/data/local/tmp` — networks are staged outside the kernel tree
`CLUSTER_MAX_GHZ`	`{"gold": 3.6288, "prime": 4.3968}` — the top of each cluster's published OPP ladder
`CLOCK_FLAG_RATIO`	`0.80` — below this fraction of the ceiling, `below_max_clock` is set
`MAX_HW_COUNTERS`	`12` — PMU counters per core; beyond this simpleperf multiplexes
default `timeout`	180.0 seconds
default `core`	mask `f0`, cores 4–7
default `power()` window	35 s, against a 30 s validated minimum